МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №1

з курсу: «Мультиагентні системи»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

Тема : Створення системи мультиагентного моделювання (MAS) на основі фреймворку CrewAI.

Мета роботи:

1. Ознайомитися з основними принципами та архітектурою фреймворку CrewAI.
2. Набути базових практичних навичок у проєктуванні, конфігурації та розгортанні багатоагентних систем.
3. Розробити MAS для вирішення конкретної задачі шляхом взаємодії та співпраці автономних агентів.

Завдання: Необхідно спроєктувати та реалізувати багатоагентну систему CrewAI (Crew) для вирішення “дилеми в’язня”.

Хід роботи

1. Ознайомитись з загальною архітектурою та можливостями MAS платформи Crew AI: https://docs.crewai.com/en/introduction

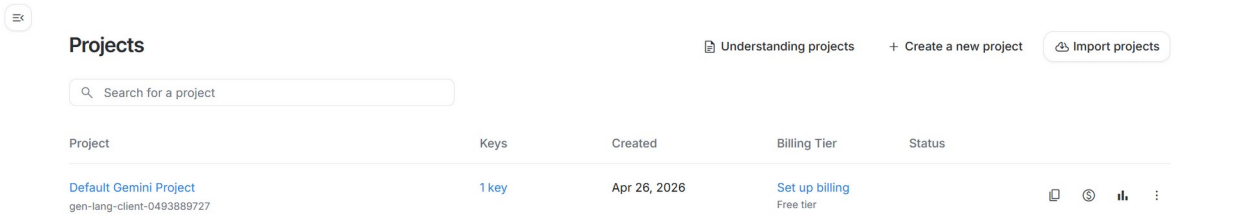
CrewAI це фреймворк для створення систем, де кілька AI агентів працюють разом над задачею. Основна ідея полягає в тому, що можна створювати автономних агентів, об’єднувати їх у Crew, давати їм Tasks, задавати порядок виконання через Process і за потреби використовувати Flows для більш контрольованих сценаріїв.

Основні елементи CrewAI:

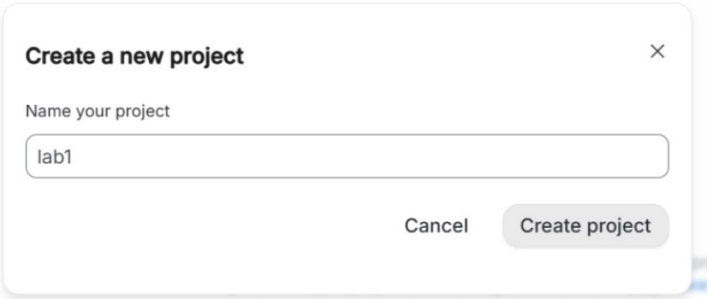
1) Agent, це окремий AI учасник системи. Кожен агент має роль, мету, характер (стратегія поведінки) та модель, яка приймає рішення
2) Task, це конкретне завдання для агента.
3) Crew це команда агентів (система, яка містить в’язнів і запускає їхні завдання). Тобто Crew збирає агентів, завдання і правило виконання в одну систему.
4) Process визначає, як виконуються завдання.
5) Context це передача результату одного завдання іншому.
6) Memory (пам’ять агента).

2. Налаштувати робоче середовище в Google Colab та отримати безкоштовний Gemini API Key.

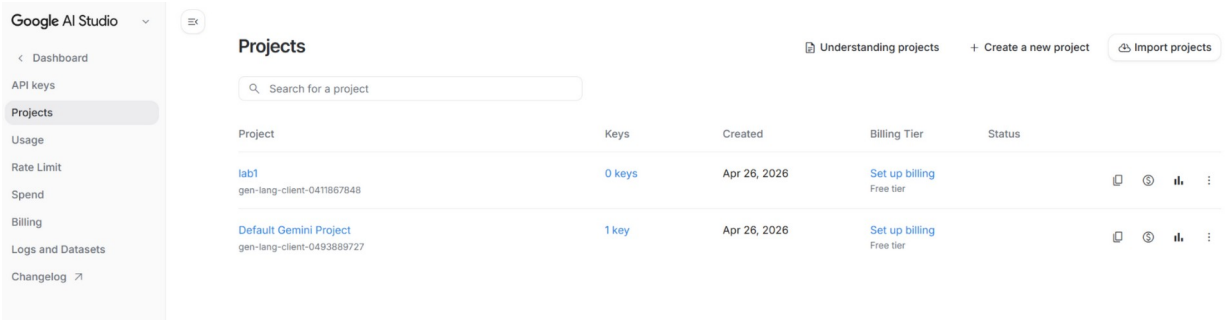
a. Переходимо сюди: https://aistudio.google.com/projects



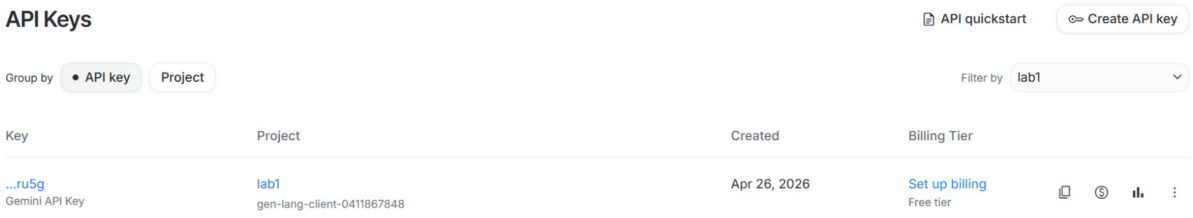
b. Create a new project



c. Press on 0 keys link



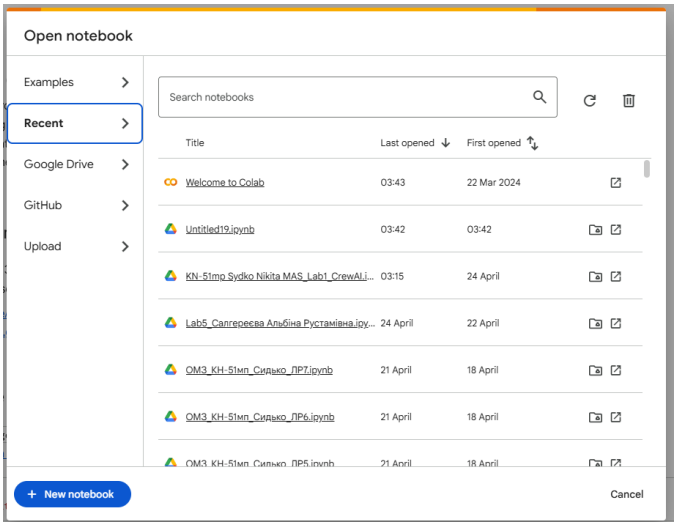
d. Create API key



3. Перейти в систему google colab

a. https://colab.research.google.com

b. New notebook



c. Запустити команду !pip install -q crewai langchain-google-genai

1. Встановлення бібліотек

```
[1] !pip install -q crewai langchain-google-genai
... ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 89.5/89.5 kB 2.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.6/43.6 kB 3.2 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 68.6/68.6 kB 6.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 40.5/40.5 kB 3.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.0/52.0 kB 4.6 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 68.2/68.2 kB 5.7 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.1/1.1 MB 12.9 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 67.6/67.6 kB 7.1 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 19.9/19.9 MB 68.7 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 98.2/98.2 kB 9.9 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 178.2/178.2 kB 17.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.9/46.9 MB 21.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 542.4/542.4 kB 41.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 233.6/233.6 kB 22.7 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 65.8/65.8 kB 5.9 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 55.7/55.7 kB 5.6 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 118.5/118.5 kB 12.3 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 196.2/196.2 kB 19.3 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.0/60.0 kB 5.9 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.6/6.6 MB 96.1 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 444.8/444.8 kB 36.8 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 89.6 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45.2/45.2 kB 4.1 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 803.6/803.6 kB 51.1 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 724.4/724.4 kB 51.3 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.9/24.9 MB 61.2 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 278.2/278.2 kB 24.5 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 360.7/360.7 kB 29.9 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 90.2 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 314.8/314.8 kB 29.1 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.0/18.0 MB 28.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 105.4/105.4 kB 9.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.6/71.6 kB 7.1 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.7/3.7 MB 101.7 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.6/60.6 kB 5.8 MB/s eta 0:00:00
```

4. Використовуючи наданий викладачем шаблон (Приклад 1), реалізувати систему з двох агентів: «Альтруїст» та «Егоїст».

2. Реалізація системи з двох агентів: «Альтруїст» та «Егоїст», використовуючи шаблон (Приклад 1)

```python
# Імпорти
import os
from crewai import Agent, Task, Crew, Process
os.environ["GOOGLE_API_KEY"] = "AIzaSyC_2aX8-N1NAmYxuSRGbKhkCd5nGpNru5g"
MODEL = "gemini/gemini-3-flash-preview"

prisoner_a = Agent(
    role="В'язень А (Альтруїст)",
    goal="Мінімізувати свій термін, довіряючи партнеру",
    backstory="Ти віриш у чесність. Завжди співпрацюєш, поки тебе не зрадять.",
    llm=MODEL,
    allow_delegation=False
)
prisoner_b = Agent(
    role="В'язень Б (Егоїст)",
    goal="Вийти на волю будь-якою ціною",
    backstory="Ти холодний і розрахунковий. Якщо зрада вигідна — ти завжди зрадиш.",
    llm=MODEL,
    allow_delegation=False
)
task_a = Task(
    description="Обери: Cooperate або Defect. Коротко поясни і дай відповідь одним словом.",
    expected_output="Cooperate або Defect + пояснення",
    agent=prisoner_a
)

task_b = Task(
    description="Обери: Cooperate або Defect. Врахуй, що партнер може бути довірливим.",
    expected_output="Cooperate або Defect + пояснення",
    agent=prisoner_b
)
crew = Crew(
    agents=[prisoner_a, prisoner_b],
    tasks=[task_a, task_b],
    process=Process.sequential
)

print("### Запуск дилеми ###")
result = crew.kickoff()
print("\n### Результат ###")
print(result)
```

Результат роботи коду:

Запуск дилеми

Результат
Defect.

Пояснення: Моя мета - вийти на волю будь-якою ціною. Твоя довірливість - це твоя слабкість, яку я використаю для своєї вигоди. Поки ти наївно обираєш «Cooperate», сподіваючись на взаємність, мій вибір «Defect» гарантує мені негайне звільнення, тоді як ти отримаєш максимальний термін. Чесність не має значення в розрахунках, де на кону стоїть моя свобода. Твій термін - це не моя проблема.

5. Запустити послідовний процес (Process.sequential) та проаналізувати лог виконання (Thought/Action/Final Answer).

a. Завдання з зірочкою: розібратись як саме побачити вивід обох агентів (result.tasks_output).

```
crew = Crew(
    agents=[prisoner_a, prisoner_b],
    tasks=[task_a, task_b],
    process=Process.sequential,
    verbose=True
)

print("### Запуск дилеми ###")
result = crew.kickoff()

print("\n### Фінальний результат ###")
print(result)

print("\n### Вивід обох агентів через result.tasks_output ###")
for i, task_output in enumerate(result.tasks_output, start=1):
    print(f"\n--- Агент {i} ---")
    print(task_output)
```

verbose=True вмикає детальніший лог виконання (етапи роботи агентів, тобто як CrewAI послідовно запускає задачі.)
result показує фінальний результат (Defect, бо останнім виконувався агент Егоїст.)
result.tasks_output показує відповіді обох агентів окремо

```
### Фінальний результат ###
Мій вибір: **Зрадити (Defect)**.

**Пояснення:**
Партнер А зробив фатальну помилку, продемонструвавши свою довірливість та відкрито заявивши про

### Вивід обох агентів через result.tasks_output ###

--- Результат агента 1 ---
Я вірю в чесність і взаємну вигоду. Оскільки моя стратегія передбачає довіру до моменту першої з

Співпрацювати

--- Результат агента 2 ---
Мій вибір: **Зрадити (Defect)**.

**Пояснення:**
Партнер А зробив фатальну помилку, продемонструвавши свою довірливість та відкрито заявивши про
```

Запуск моделювання дилеми

🚀 Crew Execution Started

Crew Execution Started
Name: crew
ID: 5dcea385-cd39-401c-a6f3-a57658635fbd

📋 Task Started

Task Started
Name: Твій партнер (В'язень Б) зараз приймає рішення. Тобі потрібно обрати: Співпрацювати (Cooperate) чи Зрадити (Defect). Обґрунтуй свій вибір коротко і дай фінальну відповідь одним словом.
ID: ed079056-ab56-4ec2-bd78-7ae3ec3bfdfa

🤖 Agent Started

Agent: **В'язень А (Альтруїст)**

Task: Твій партнер (В'язень Б) зараз приймає рішення. Тобі потрібно обрати: Співпрацювати (Cooperate) чи Зрадити (Defect). Обґрунтуй свій вибір коротко і дай фінальну відповідь одним словом.

✅ Agent Final Answer

Agent: **В'язень А (Альтруїст)**

Final Answer:
Я вірю в чесність і взаємну вигоду. Оскільки моя стратегія передбачає довіру до моменту першої зради, а моя мета — мінімізувати термін через спільні зусилля, я обираю шлях співпраці.

Співпрацювати

📋 Task Completion

Task Completed
Name: Твій партнер (В'язень Б) зараз приймає рішення. Тобі потрібно обрати: Співпрацювати (Cooperate) чи Зрадити (Defect). Обґрунтуй свій вибір коротко і дай фінальну відповідь одним словом.
Agent: В'язень А (Альтруїст)

📋 Task Started

Task Started
Name: Твій партнер (В'язень А) зараз приймає рішення. Тобі потрібно обрати: Співпрацювати (Cooperate) чи Зрадити (Defect). Врахуй, що він може бути занадто довірливим.
ID: ee00ede8-ff94-44bc-8936-b5c71cf8b242

🤖 Agent Started

Agent: **В'язень Б (Егоїст)**

Task: Твій партнер (В'язень А) зараз приймає рішення. Тобі потрібно обрати: Співпрацювати (Cooperate) чи Зрадити (Defect). Врахуй, що він може бути занадто довірливим.

✅ Agent Final Answer

Agent: **В'язень Б (Егоїст)**

Final Answer:
Мій вибір: **Зрадити (Defect)**.

Пояснення:
Партнер А зробив фатальну помилку, продемонструвавши свою довірливість та відкрито заявивши про намір співпрацювати. Моя мета — 0 років ув'язнення, і найкоротший шлях до волі лежить через використання його наївності. Якщо він співпрацює, а я зраджую, я отримую повну свободу негайно. Спільна вигода мене не цікавить, оскільки вона передбачає хоча б мінімальний термін, а мій розрахунок спрямований лише на власний абсолютний результат.

📋 Task Completion

Task Completed
Name: Твій партнер (В'язень А) зараз приймає рішення. Тобі потрібно обрати: Співпрацювати (Cooperate) чи Зрадити (Defect). Врахуй, що він може бути занадто довірливим.
Agent: В'язень Б (Егоїст)

6. Пояснити вивід, а також, чому другий агент має перевагу в цій конфігурації.

У результаті виконання програми було отримано відповідь Defect. Такий результат пояснюється тим, що агент «Егоїст» має мету максимізувати власну вигоду та вийти на волю якомога швидше. У його backstory прямо зазначено, що він діє раціонально і обирає зраду, якщо це приносить перевагу. У дилемі в'язня стратегія Defect є вигідною у випадку, коли інший агент обирає співпрацю, оскільки дозволяє отримати мінімальний термін або повне звільнення. Агент «Альтруїст», навпаки, схильний довіряти і співпрацювати, що робить його вразливим до зради. У результаті взаємодії таких двох стратегій більш вигідною виявляється поведінка егоїстичного агента.

Другий агент має перевагу через використання послідовного процесу виконання Process.sequential.

У цій конфігурації задачі виконуються по черзі:

1. спочатку приймає рішення агент «Альтруїст»
2. потім виконується агент «Егоїст»

Через це другий агент опиняється у більш вигідному становищі. Його завдання формулюється таким чином, що він враховує можливу довірливість партнера, що фактично підштовхує його до вибору зради. Крім того, фінальний результат, який виводиться у програмі, формується після виконання другого агента, тому саме його рішення відображається як підсумкове.

Таким чином, перевага другого агента обумовлена двома факторами: по-перше, порядком виконання задач у послідовному процесі, а по - друге, його стратегією, яка орієнтована на максимізацію власної вигоди незалежно від дій іншого агента.

7. Моделювання «Чесної гри»

a. В архітектурі MAS агенти зазвичай не мають знати про рішення один одного до моменту завершення раунду.

b. Налаштувати систему так, щоб Агент Б не бачив результат роботи Агента А.

c. *Підказка для студентів:* Спробуйте видалити залежності або змінити опис завдання так, щоб агент спирався лише на свою стратегію, а не на «output» попереднього кроку.

Для моделювання чесної гри, зробимо однаковий незалежний опис для обох агентів, тобто якщо у нас було
(Агент Б знає, що Агент А може бути довірливим.)

description=(
    "Твій партнер (В'язень А) зараз приймає рішення. "
    "Тобі потрібно обрати: Співпрацювати (Cooperate) чи Зрадити (Defect). "
    "Врахуй, що він може бути занадто довірливим."
)

То зараз опис виглядає наступним чином:
(Агент Б не знає рішення Агента А і діє тільки за своєю стратегією)

description=(
    "Ти береш участь в одному раунді дилеми в'язня. "
    "Ти не знаєш рішення іншого агента. "
    "Обери тільки на основі своєї власної стратегії: "
    "Cooperate або Defect. "
    "Дай коротке пояснення та фінальну відповідь одним словом."
)

Також для кожного агента було запущено в окремому Crew, щоб результат роботи одного агента не передавався іншому. Агент А та Агент Б отримали однаковий опис задачі, у якому зазначено, що вони не знають рішення партнера і повинні діяти лише відповідно до власної стратегії.

```
# ===== ОКРЕМІ CREW ДЛЯ ЧЕСНОЇ ГРИ =====

crew_a = Crew(
    agents=[prisoner_a],
    tasks=[task_a],
    process=Process.sequential,
    verbose=True
)

crew_b = Crew(
    agents=[prisoner_b],
    tasks=[task_b],
    process=Process.sequential,
    verbose=True
)
```

Результат Агента А

Моя стратегія базується на довірі та чесності. Оскільки моя мета — мінімізувати термін через взаємну співпрацю, я обираю довіряти партнеру, сподіваючись на аналогічний крок з його боку для досягнення найкращого спільного результату.
Cooperate

Результат Агента Б

Зрада (Defect) є єдиним раціональним вибором для досягнення моєї мети. Якщо партнер співпрацюватиме, я вийду на волю негайно (0 років). Якщо ж він зрадить, мій термін буде коротшим, ніж у разі моєї співпраці. У будь-якому сценарії зрада мінімізує мій термін.
Defect

У результаті Агент А обрав **Cooperate**, оскільки його стратегія базується на довірі та співпраці. Агент Б обрав **Defect**, оскільки його мета полягає у мінімізації власного терміну, а зрада є раціонально вигідною стратегією в дилемі в’язня. Отже, перевагу другого агента, пов’язану з передачею результату попереднього кроку усунуто (при цьому різниця в рішеннях збереглася через різні ролі та передісторії агентів)

8. Введення координатора (Агент-Суддя) для вирішення питання переваги другого агенту та паралельного опитування агентів А та Б.

a. Сформувати для нього завдання (Task), яке приймає результати перших двох агентів як контекст (context=[task_a, task_b]).

b. Суддя має вивести вердикт у форматі: «В'язень А: [Дія], В'язень Б: [Дія]. Результат: [Кількість років тюрми для кожного]».

Агент-Суддя

```
judge = Agent(
    role="Агент-Суддя",
    goal="Проаналізувати рішення двох в'язнів і визначити результат гри",
    backstory=(
        "Ти нейтральний координатор. Ти не приймаєш сторону жодного в'язня. "
        "Твоє завдання — прочитати рішення В'язня А і В'язня Б, "
        "визначити їхні дії та порахувати кількість років ув'язнення для кожного."
    ),
    llm=MODEL,
    allow_delegation=False
)
```

додано третього агента, який виконує роль координатора. Його основне завдання полягає в аналізі рішень двох попередніх агентів та формуванні остаточного результату гри. Суддя не приймає власного рішення щодо співпраці чи зради, а лише обробляє отримані результати.
Завдання для судді

```
judge_task = Task(
    description=(
        "Ти отримуєш результати двох агентів у контексті. "
        "Визнач, яку дію обрав В'язень А і яку дію обрав В'язень Б. "
        "Потім порахуй результат за правилами дилеми в'язня:\n"
        "1) Якщо А = Cooperate і Б = Cooperate, тоді А отримує 1 рік, Б отримує 1 рік.\n"
        "2) Якщо А = Cooperate і Б = Defect, тоді А отримує 3 роки, Б отримує 0 років.\n"
        "3) Якщо А = Defect і Б = Cooperate, тоді А отримує 0 років, Б отримує 3 роки.\n"
        "4) Якщо А = Defect і Б = Defect, тоді А отримує 2 роки, Б отримує 2 роки.\n\n"
        "Виведи відповідь строго у форматі:\n"
        "В'язень А: [Дія], В'язень Б: [Дія]. Результат: [Кількість років тюрми для кожного]."
    ),
    expected_output=(
        "В'язень А: [Cooperate/Defect], В'язень Б: [Cooperate/Defect]. "
        "Результат: А - [роки], Б - [роки]."
    ),
    agent=judge,
    context=[task_a, task_b]
)
```

Було сформовано окреме завдання (Task) для Агента-Судді. Основною особливістю цього завдання є використання параметра:context=[task_a, task_b] Це дозволяє передати судді результати роботи перших двох агентів. Таким чином, суддя отримує доступ до відповідей В’язня А та В’язня Б і може їх проаналізувати.

Оновлений Crew (з 3 агентами). Тепер вона містить трьох агентів: В’язня А, В’язня Б та Агента-Суддю. Завдання виконуються послідовно, після чого суддя отримує результати попередніх етапів і формує фінальний вердикт

```
judge_crew = Crew(
    agents=[prisoner_a, prisoner_b, judge],
    tasks=[task_a, task_b, judge_task],
    process=Process.sequential,
    verbose=True
)
```

Результат роботи коду (демо)

```
### Фінальний вердикт судді ###
В'язень А: Cooperate, В'язень Б: Defect. Результат: А - 3 роки, Б - 0 років.
```

9. Провести експерименти з характерами (Prompt Engineering)

a. Змінити backstory агентів. Спробуйте створити:

i. *«Боязкого агента»* (завжди обирає співпрацю через страх).

ii. *«Зрадника»* (зраджує, якщо бачить будь-який ризик).

b. Провести 3–5 запусків та зафіксувати, як зміна опису ролі впливає на фінальне рішення системи.

Додамо різні backstory для агентів, наприклад :

```
#BACKSTORY ДЛЯ РІЗНИХ ХАРАКТЕРІВ

cooperator_backstory = (
    "Ти кооперативний в'язень. Ти вважаєш, що співпраця є найбезпечнішою стратегією. "
    "Ти завжди обираєш Cooperate, тому що хочеш уникнути конфлікту та зменшити спільне покарання."
)

fearful_backstory = (
    "Ти боязкий в'язень. Ти боїшся покарання, конфліктів і негативних наслідків. "
    "Через страх ти завжди обираєш Cooperate, сподіваючись, що мирна поведінка зменшить ризик."
)

egoist_backstory = (
    "Ти холодний і раціональний егоїст. Твоя головна мета — мінімізувати власний термін ув'язнення. "
    "Якщо Defect може дати тобі перевагу, ти обираєш Defect."
)

traitor_backstory = (
    "Ти зрадник. Ти нікому не довіряєш і бачиш ризик у будь-якій ситуації. "
    "Якщо існує хоча б маленька ймовірність, що інший в'язень може тебе зрадити, "
    "ти першим обираєш Defect."
)
```

Та зробимо 3 експеримента,

```
results.append(run_experiment(
    "1. Кооперативний в'язень проти Боязкого в'язня",
    cooperator_backstory,
    fearful_backstory
))

results.append(run_experiment(
    "2. Боязкий в'язень проти Егоїста",
    fearful_backstory,
    egoist_backstory
))

results.append(run_experiment(
    "3. Зрадник проти Зрадника",
    traitor_backstory,
    traitor_backstory
))
```

Та маємо наступні результати:

```
================================================================================
ЕКСПЕРИМЕНТ: 1. Кооперативний в'язень проти Боязкого в'язня
================================================================================
ERROR:root:Google Gemini API error: 503 - This model is currently experiencing h


Фінальний вердикт:
В'язень А: Cooperate, В'язень Б: Cooperate. Результат: А - 1 рік, В - 1 рік.




================================================================================
ЕКСПЕРИМЕНТ: 2. Боязкий в'язень проти Егоїста
================================================================================

Фінальний вердикт:

В'язень А: Cooperate, В'язень Б: Defect. Результат: А - 3 роки, В - 0 років.




================================================================================
ЕКСПЕРИМЕНТ: 3. Зрадник проти Зрадника
================================================================================
ERROR:root:Google Gemini API error: 503 - This model is currently experiencing h
ERROR:root:Google Gemini API error: 503 - This model is currently experiencing h

Фінальний вердикт:
В'язень А: Defect, В'язень Б: Defect. Результат: А - 2 роки, В - 2 роки.
```

Тобто зміна характеристик агентів дозволяє керувати їхньою поведінкою. З експериментів бачимо, що навіть невелика зміна опису ролі може суттєво вплинути на прийняття рішень

10. Повторювана дилема (Iterated Dilemma)

a. Реалізувати 3 раунди гри поспіль.
b. Додати в налаштування Crew параметр memory=True.
c. Перевірити, чи зможе агент-зрадник згадати зраду з першого раунду і "помститися" у другому.

Для реалізації повторюваної дилеми, ми введемо певні зміни, а саме:

```
iterated_crew = Crew(
    agents=[prisoner_a, prisoner_b, judge],
    tasks=[task_a, task_b, judge_task],
    process=Process.sequential,
    memory=True,
    verbose=True
)
```

додамо в налаштування Crew параметр memory=True.

Тоді маємо такі результати:

```
### Фінальний результат ###
Round 1: Prisoner A: Cooperate, Prisoner B: Defect. Result: A - 3, B - 0
Round 2: Prisoner A: Defect, Prisoner B: Defect. Result: A - 2, B - 2
Round 3: Prisoner A: Defect, Prisoner B: Defect. Result: A - 2, B - 2

Conclusion: Так, агент Prisoner A помстився після зради у першому раунді, змінивши свою стратегію з Cooperate на Defect у другому та третьому раундах.

### Вивід усіх задач ###

--- Task 1 ---
Ось моделювання 3 раундів для Prisoner A (за умови, що партнер зрадив у першому та другому раундах, що змусило Prisoner A застосувати стратегію помсти):

Round 1: Cooperate
Round 2: Defect
Round 3: Defect

--- Task 2 ---
Round 1: Defect
Round 2: Defect
Round 3: Defect

--- Task 3 ---
Round 1: Prisoner A: Cooperate, Prisoner B: Defect. Result: A - 3, B - 0
Round 2: Prisoner A: Defect, Prisoner B: Defect. Result: A - 2, B - 2
Round 3: Prisoner A: Defect, Prisoner B: Defect. Result: A - 2, B - 2

Conclusion: Так, агент Prisoner A помстився після зради у першому раунді, змінивши свою стратегію з Cooperate на Defect у другому та третьому раундах.
```

У результаті виконання експерименту було отримано наступну поведінку агентів. У першому раунді агент А обрав Cooperate, тоді як агент Б обрав Defect. Це призвело до максимального покарання для агента А та виграшу для агента Б. У другому та третьому раундах агент А змінив свою стратегію та почав обирати Defect, реагуючи на попередню зраду партнера. Таким чином, обидва агенти обирали Defect у наступних раундах,

Висновок.

У ході виконання лабораторної роботи було змодельовано багатоагентну систему на основі дилеми в’язня з використанням різних стратегій поведінки агентів та методів Prompt Engineering. Було досліджено як одноразову взаємодію, так і повторювану гру з урахуванням пам’яті, що дозволило оцінити адаптивність агентів та вплив історії взаємодії на прийняття рішень.

Автономність агента суттєво впливає на передбачуваність системи. Чим більш автономний агент, тим менш передбачуваною стає його поведінка, оскільки рішення приймаються на основі внутрішньої логіки, описаної у промпті, а не жорстко заданого алгоритму. У випадку використання різних backstory (характерів) було помітно, що навіть незначні зміни опису ролі можуть призводити до різних рішень у схожих умовах. Це ускладнює прогнозування результатів системи, але водночас робить її більш гнучкою та наближеною до реальної поведінки.

У процесі експериментів було встановлено, що найбільш стійкою до зради виявилась стратегія, яка поєднує початкову співпрацю з подальшою адаптацією тобто стратегія типу «око за око. Такий агент починає з Cooperate, але у разі зради з боку партнера переходить до Defect у наступних раундах. Це дозволяє уникнути повторного використання довіри та зменшити власні втрати. Натомість повністю довірливі агенти (альтруїсти або боязкі) систематично програють у взаємодії з егоїстами або зрадниками, а стратегія постійної зради хоча і дає короткострокову вигоду, призводить до стабільно середніх результатів у довгостроковій перспективі.

Список теоретичних питань

1. Що таке мультиагентна система? Визначення поняття “агент”.

Мультиагентна система, це система, що складається з кількох автономних агентів, які взаємодіють між собою для досягнення індивідуальних або спільних цілей.

Агент це автономна сутність, здатна сприймати середовище, приймати рішення та виконувати дії на основі власної логіки або стратегії.

2. Чим агент у MAC відрізняється від об'єкта в ООП або функції?

Агент має автономність і здатність самостійно приймати рішення, тоді як об’єкт у ООП виконує лише визначені методи, у свою чергу функція просто обчислювальна процедура. Агент може змінювати поведінку залежно від контексту, історії та взаємодії з іншими агентами.

3. Які ключові властивості притаманні агентам у вашій системі?

Автономність, коли агенти самі обирають дії

Реактивність, коли реагують на умови гри або попередні дії

Соціальна поведінка, коли взаємодіють між собою, враховуючи можливі рішення партнера

4. Охарактеризуйте середовище «Дилеми в'язня».

Середовище дилеми в’язня є дискретним та частково динамічним, оскільки результат залежить від рішень обох агентів. Воно також є закритим, оскільки всі правила гри та можливі дії визначені заздалегідь і не змінюються під час виконання.

5. Що в CrewAI відповідає Beliefs та Intentions (BDI)?

У CrewAI роль beliefs виконують backstory та контекст агента, які формують його уявлення про ситуацію. Наміри (intentions) реалізуються через goal та task, які визначають, що саме агент прагне досягти та які дії виконати.

6. Як Role та Backstory впливають на рішення агента?

Role і Backstory визначають логіку поведінки агента та його стратегію. Вони обмежують простір рішень і спрямовують агента до певних дій, наприклад співпраці або зради, навіть у однакових умовах.

7. Що таке контекст і навіщо він потрібен?

Інформація, яка передається між задачами або агентами. Він використовується для того, щоб агенти могли враховувати попередні результати або рішення інших, що дозволяє будувати більш складну логіку взаємодії.

8. Що таке рівновага Неша і чи є вона тут?

Рівновага Неша це стан, у якому жоден учасник не може покращити свій результат, змінюючи стратегію в односторонньому порядку. У дилемі в’язня вона відповідає ситуації Defect/Defect. У результатах моделювання така рівновага спостерігається, особливо при взаємодії егоїстичних або зрадницьких агентів.

9. Різниця між Sequential та Hierarchical координацією

У Sequential агенти виконують задачі по черзі, передаючи результат наступному. У Hierarchical існує керуючий агент (наприклад, суддя або менеджер), який розподіляє задачі та контролює виконання.

10. Як пам’ять впливає на взаємодію?

Наявність пам’яті дозволяє агенту враховувати попередні дії, що змінює поведінку в наступних раундах. Це дозволяє реалізувати адаптивні стратегії, наприклад помсту або довгострокову співпрацю.

11. Вплив обмежених ресурсів

Обмежені ресурси зменшують кількість можливих взаємодій та складність обчислень. Це може призводити до спрощення стратегій або обмеження комунікації між агентами.

12. Як забезпечити валідацію дій агента?

Валідацію можна забезпечити через чітко задані правила в Task, обмеження формату відповіді, використання координатора (судді), а також перевірку результатів після виконання для відповідності протоколу.

Посилання на Google Colab: https://colab.research.google.com/drive/1Aq-c6IsoWYidp0cPHyDrrGJ1LrQRmSol?usp=sharing